



CLICKGEO

CURSOS DE GEOTECNOLOGIAS

Análise Geoespacial em R Part 2

Para trabalhar com dados raster em R, utilizamos o pacote raster, que oferece uma ampla gama de funções para a criação, leitura, manipulação e processamento de dados raster. Este pacote permite trabalhar eficientemente com grandes conjuntos de dados raster que podem ser grandes demais para caber na memória principal do computador. Em tais casos, o pacote raster oferece a possibilidade de processar os dados em partes menores, evitando a necessidade de carregar o arquivo raster completo na RAM.

```
# Instalar biblioteca
#install.packages("raster", dependencies = TRUE)
#install.packages("sf", dependencies = TRUE)
#install.packages("terra", dependencies = TRUE)
#install.packages("marmap", dependencies = TRUE)
#install.packages("tmap", dependencies = TRUE)
#install.packages("tidyverse")
#install.packages("viridis")
```

```
# Importar bibliotecas
library(raster)
```

```
## Loading required package: sp
```

```
library(ggplot2)
library(sf)
```

```
## Linking to GEOS 3.11.2, GDAL 3.8.2, PROJ 9.3.1; sf_use_s2() is TRUE
```

```
library(terra)
```

```
## terra 1.7.71
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:terra':
##
## intersect, union
```

```
## The following objects are masked from 'package:raster':
##
## intersect, select, union
```

```
## The following objects are masked from 'package:stats':
##
## filter, lag
```

```
## The following objects are masked from 'package:base':
##
## intersect, setdiff, setequal, union
```

```
library(tmap)
```

```
## Breaking News: tmap 3.x is retiring. Please test v4, e.g. with
## remotes::install_github('r-tmap/tmap')
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v forcats 1.0.0 v stringr 1.5.1
## v lubridate 1.9.3 v tibble 3.2.1
## v purrr 1.0.2 v tidyr 1.3.1
## v readr 2.1.5
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x tidyr::extract() masks terra::extract(), raster::extract()
## x dplyr::filter() masks stats::filter()
## x dplyr::lag() masks stats::lag()
## x dplyr::select() masks raster::select()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(readr)
library(viridis)
```

```
## Loading required package: viridisLite
```

```
unzip("D:/Pos-GeoDataScience/M2-R/dado_pratica/07S36_ZN.zip", exdir = "D:/Pos-GeoDataScience/M2-R/dado_pratica/07S36_ZN")
```

```
# Carregar dados geoespaciais
municipios <- st_read("D:/Pos-GeoDataScience/M2-R/dado_pratica/PB_Municipios/PB_Municipios_2022.shp")
```

```
## Reading layer 'PB_Municipios_2022' from data source
## 'D:\Pos-GeoDataScience\M2-R\dado_pratica\PB_Municipios\PB_Municipios_2022.shp'
## using driver 'ESRI Shapefile'
```

```
## Simple feature collection with 223 features and 4 fields
## Geometry type: MULTIPOLYGON
## Dimension:      XY
## Bounding box:   xmin: -38.7656 ymin: -8.302955 xmax: -34.79309 ymax: -6.025912
## Geodetic CRS:   SIRGAS 2000
```

```
joaoPessoa <- municipios %>% filter(municipios$NM_MUN == 'João Pessoa')
```

```
# Ler dados raster
```

```
rasterData <- raster("D:/Pos-GeoDataScience/M2-R/dado_pratica/raster/07S36_ZN.tif", format = "GTiff")
```

```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

```
rasterData
```

```
## class      : RasterLayer
## dimensions : 3600, 5400, 19440000 (nrow, ncol, ncell)
## resolution : 0.0002777785, 0.0002777776 (x, y)
## extent      : -36, -34.5, -8, -7 (xmin, xmax, ymin, ymax)
## crs         : +proj=longlat +datum=WGS84 +no_defs
## source      : 07S36_ZN.tif
## names       : X07S36_ZN
```

```
plot(rasterData,)
```

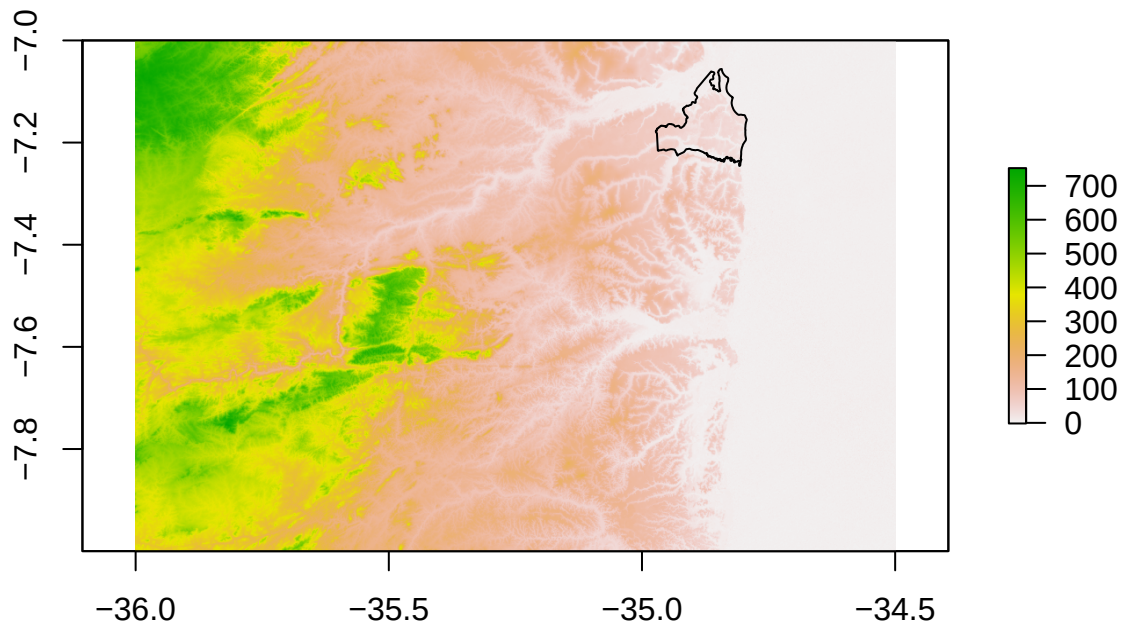
```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

```
plot(st_geometry(joaoPessoa$geometry), add = TRUE)
```



Recortar Image

Para realizar o recorte de uma imagem raster no R, utilizando a geometria de um shapefile específico, você pode seguir os seguintes passos, adaptando o procedimento realizado em Python para o ambiente do R. A função equivalente a `mask` no pacote raster do R será usada para aplicar o recorte.

```
# Recortar raster pelo polígono do município
rasterRecortado <- mask(rasterData, joaoPessoa)
```

```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

```
## Warning: Key Unknown-0 of TIFFTagLocation=0 has count=0, whereas only 1 is
## legal. (GDAL error 1)
```

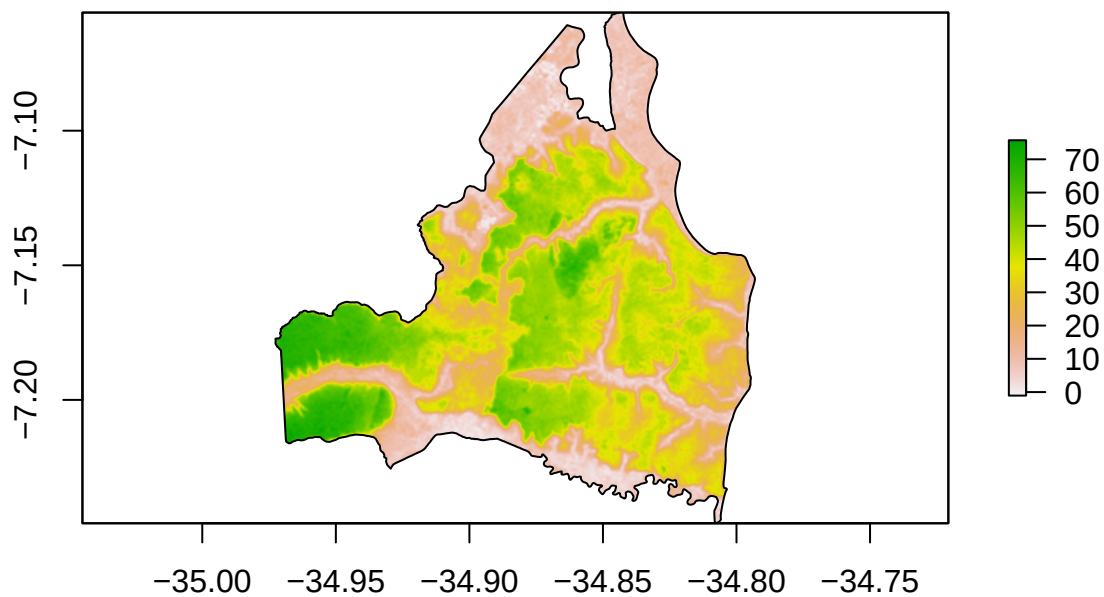
```
joaoPessoa_sp <- as(joaoPessoa, "Spatial")
```

```
# Obtendo os limites da geometria do shapefile
```

```
xmin <- bbox(joaoPessoa_sp)[1,1]
xmax <- bbox(joaoPessoa_sp)[1,2]
ymin <- bbox(joaoPessoa_sp)[2,1]
ymax <- bbox(joaoPessoa_sp)[2,2]
```

```
# Visualizar dados
```

```
plot(rasterRecortado, xlim=c(xmin, xmax), ylim=c(ymin, ymax))
plot(st_geometry(joaoPessoa), add = TRUE)
```



Fatiamento e Reclassificação

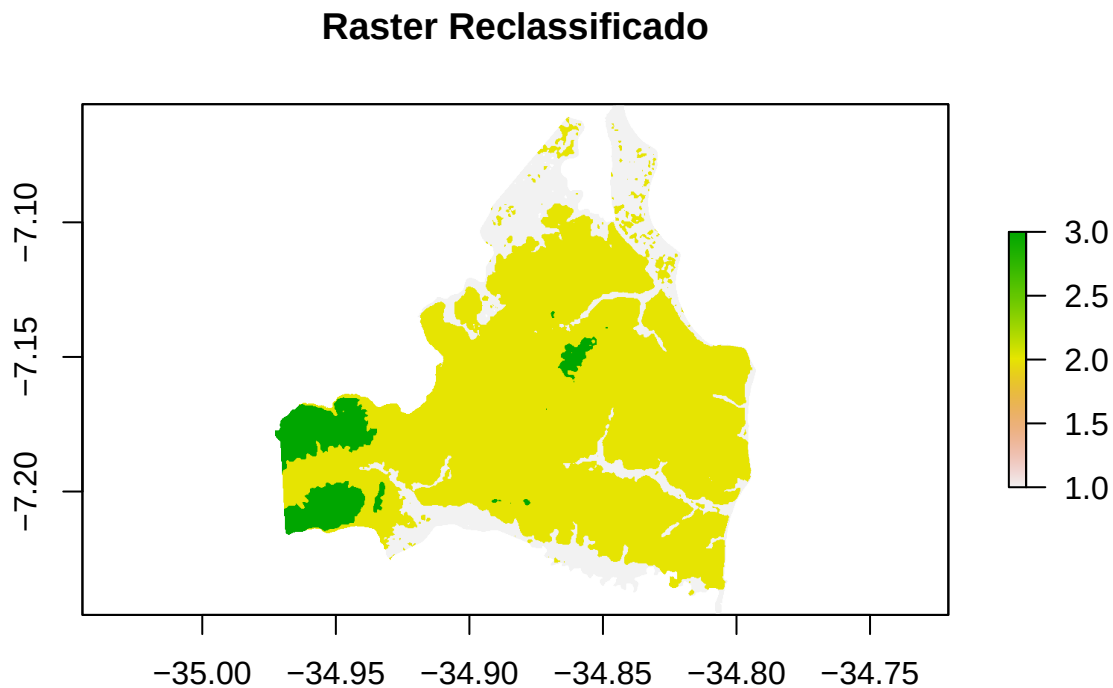
Para realizar o fatiamento e a reclassificação de um raster no R, você pode usar a função `reclassify()`. Essa função permite reclassificar os valores de um raster, transformando os valores originais em novas categorias com base em intervalos especificados.

```
# Definir as regras de reclassificação
```

```
reclassRules <- matrix(c(-Inf, 0, NA,
                          0, 10, 1,
                          10, 60, 2,
                          60, 100, 3,
                          100, Inf, NA), byrow = TRUE, ncol = 3)
```

```
# Aplicar a reclassificação
rasterReclassificado <- reclassify(rasterRecortado, reclassRules)

# Visualizar o raster reclassificado
plot(rasterReclassificado, main = "Raster Reclassificado", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
```



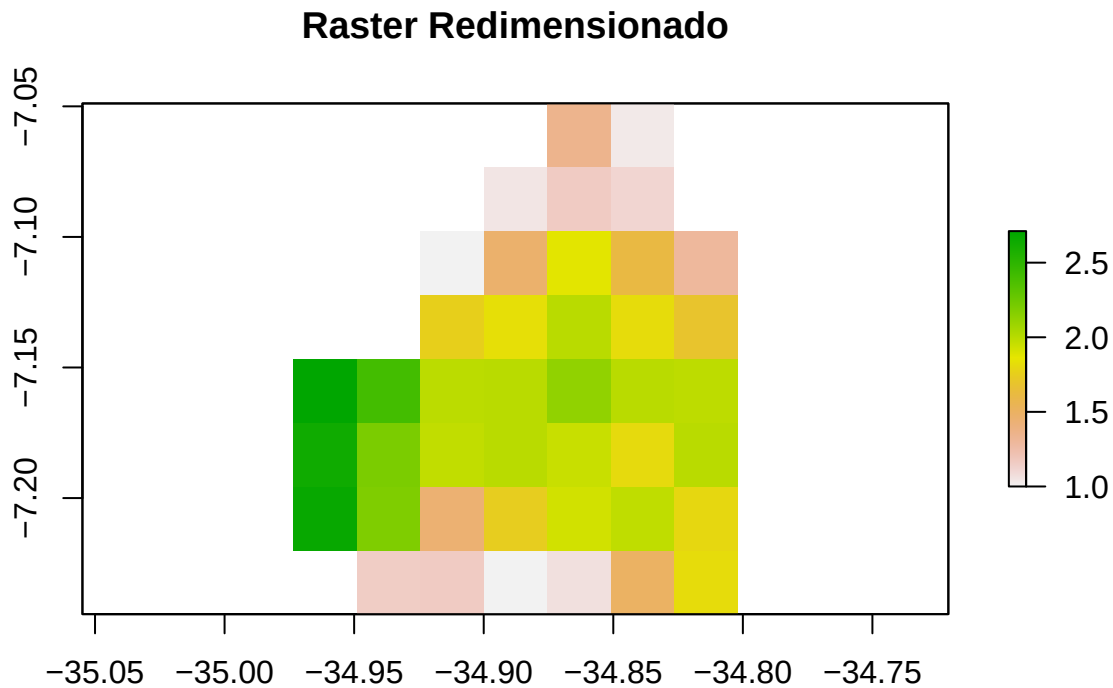
Redimensionar

Para redimensionar imagens raster no R, podemos usar o pacote raster para ajustar a resolução espacial de um raster. Embora o pacote raster não tenha uma função direta equivalente à `resize` do skimage em Python, podemos alcançar um resultado similar alterando a resolução do raster e utilizando a função `aggregate()` para aumentar a resolução ou `disaggregate()` para diminuir a resolução.

```
# Determinar o fator de agregação para alcançar aproximadamente a resolução desejada
fator <- sqrt(ncell(rasterReclassificado) / (50 * 50))

# Diminuir a resolução: Usar aggregate() para resolução mais baixa
rasterRedimensionado <- aggregate(rasterReclassificado, fact=fator)

# Visualizar o raster redimensionado
plot(rasterRedimensionado, main="Raster Redimensionado", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
```



Mudar Sistema de Referências

Também podemos realizar a Reprojeção com R, usando uma única função `projectRaster`.

```
# Definir o sistema de referências
dest_crs <- "EPSG:3395" #WGS 84 / World Mercator

# Reprojetar o raster para o novo sistema de referências
raster_reprojetado <- projectRaster(rasterReclassificado, crs=dest_crs)
```

Salvar Imagens

Todos os processamento que realizamos podem ser salvo também usando a função `writeRaster` para isso.

```
# Definir o caminho de saída para o raster reprojetado
caminho_reprojetado <- "D:/Pos-GeoDataScience/M2-R/dado_pratica/raster/07S36_ZN_PROJECT.tif"

# Salvar o raster reprojetado
writeRaster(raster_reprojetado, caminho_reprojetado, overwrite=TRUE)
```

Dividir Imagem

Subdividir uma imagem no R em bem parecido com o que vimos no Pthon. O processo segue a mesma lógica que fizemos anteriormente, onde o primeiro processo será encontrar as dimensões da nossa área de interesse

e criar em seguida as partes da imagem.

```
# Projetando regi'ao de interesse
joaoPessoa_project <- st_transform(joaoPessoa, crs = 3395)

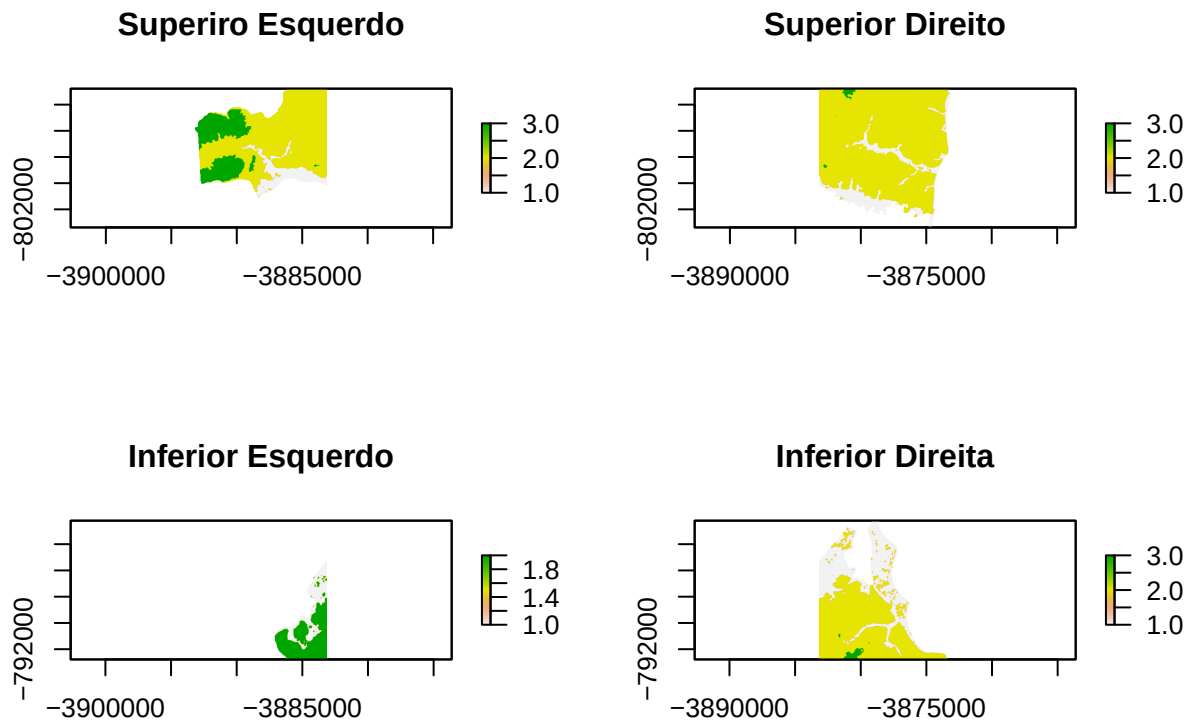
# Obter a extensão total do raster
ext_total <- extent(joaoPessoa_project)

# Calcular o centro da extensão em X e Y
x_center <- ext_total@xmin + (ext_total@xmax - ext_total@xmin) / 2
y_center <- ext_total@ymin + (ext_total@ymax - ext_total@ymin) / 2

# Criar objetos de extensão para cada quadrante
ext_se <- extent(ext_total@xmin, x_center, ext_total@ymin, y_center)
ext_sd <- extent(x_center, ext_total@xmax, ext_total@ymin, y_center)
ext_ie <- extent(ext_total@xmin, x_center, y_center, ext_total@ymax)
ext_id <- extent(x_center, ext_total@xmax, y_center, ext_total@ymax)

# Usar crop para criar cada parte
parte_superior_esquerda <- crop(raster_reprojetado, ext_se)
parte_superior_direita <- crop(raster_reprojetado, ext_sd)
parte_inferior_esquerda <- crop(raster_reprojetado, ext_ie)
parte_inferior_direita <- crop(raster_reprojetado, ext_id)

par(mfrow=c(2,2))
plot(parte_superior_esquerda, main="Superior Esquerdo")
plot(parte_superior_direita, main="Superior Direito")
plot(parte_inferior_esquerda, main="Inferior Esquerdo")
plot(parte_inferior_direita, main="Inferior Direita")
```

Mosaico

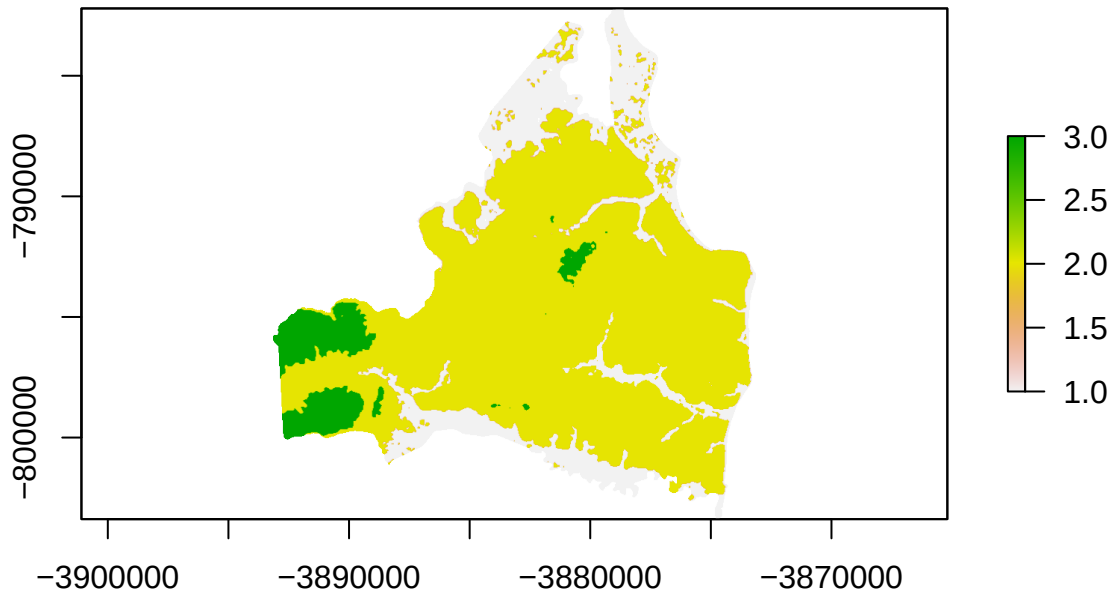
Para combinar ou criar um mosaico com múltiplos rasters o procedimento geral seria feito com a função `merge`.

```
# Exemplo usando merge para combinar duas partes
mosaico_superior <- merge(parte_superior_esquerda, parte_superior_direita)
mosaico_inferior <- merge(parte_inferior_esquerda, parte_inferior_direita)

# E depois combinando as duas metades
imagem_completa <- merge(mosaico_superior, mosaico_inferior, along = 2)

plot(imagem_completa, main = "Mosaico do Raster")
```

Mosaico do Raster



Operações Matemáticas

No ambiente da linguagem R, uma ampla gama de operações pode ser executada em dados raster com eficiência e flexibilidade, especialmente através do uso do pacote `raster`.

```
# 1. Normalização
normalized <- (rasterRecortado - minValue(rasterRecortado)) / (maxValue(rasterRecortado) - minValue(rasterRecortado))

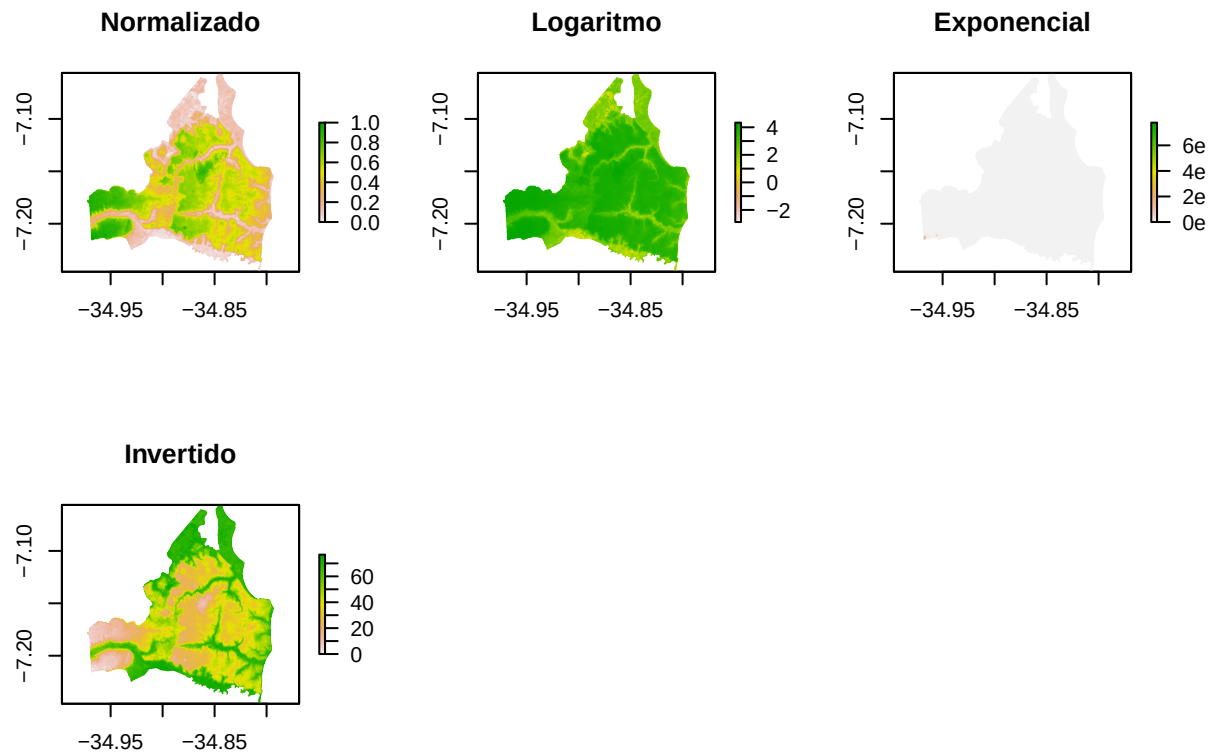
# 2. Logaritmo
log_transformed <- log1p(rasterRecortado)
```

```
## Warning in log1p(getValues(x)): NaNs produced
```

```
# 3. Exponencial
exponential_transformed <- exp(rasterRecortado)

# 4. Inversão de Valores
inverted <- maxValue(rasterRecortado) - rasterRecortado
```

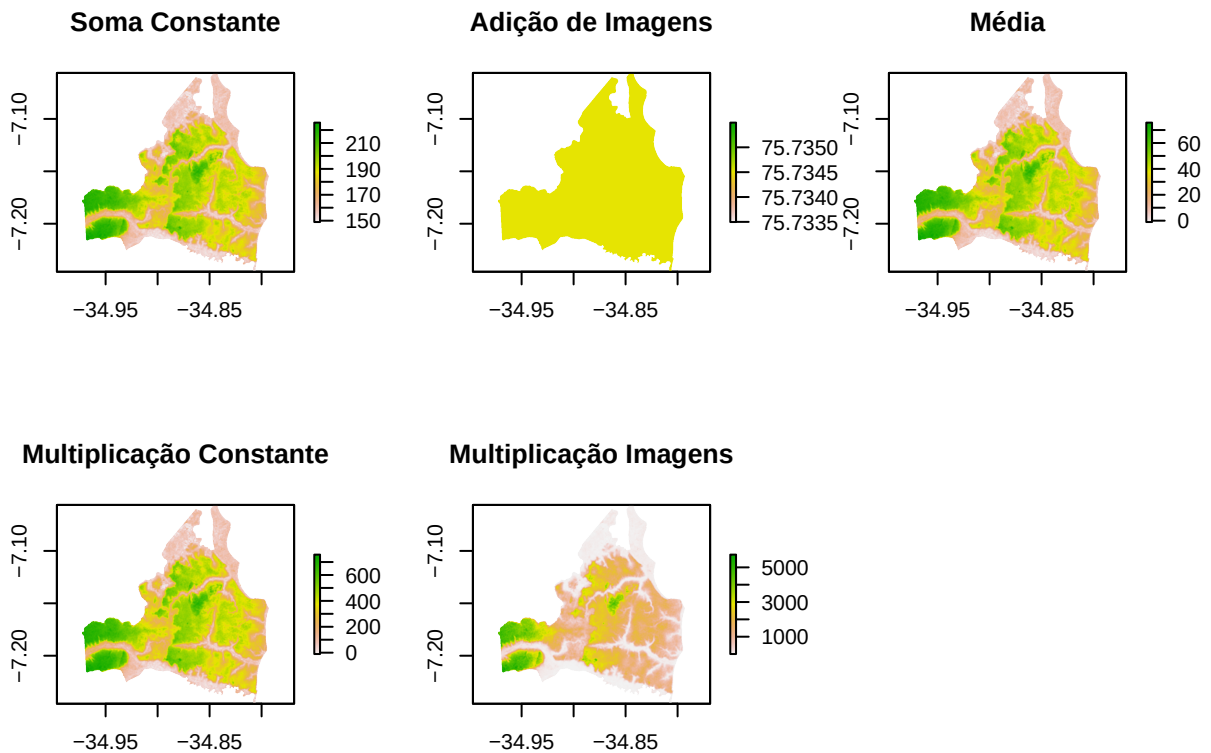
```
# Para visualização, você pode usar o par(mfrow=c(n,m)) para organizar os plots e plot para visualizar
par(mfrow=c(2,3))
plot(normalized, main="Normalizado", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
plot(log_transformed, main="Logaritmo", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
plot(exponential_transformed, main="Exponencial", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
plot(inverted, main="Invertido", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
```



Outras operações:

```
soma_constante <- rasterRecortado + 150
adicao_imagens <- rasterRecortado + inverted
media_imagens <- (rasterRecortado + rasterRecortado) / 2
multiplicacao_constante <- rasterRecortado * 10
multiplicacao_imagens <- rasterRecortado * rasterRecortado
```

```
par(mfrow=c(2,3))
plot(soma_constante, main="Soma Constante", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
plot(adicao_imagens, main="Adição de Imagens", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
plot(media_imagens, main="Média", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
plot(multiplicacao_constante, main="Multiplicação Constante", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
plot(multiplicacao_imagens, main="Multiplicação Imagens", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
```



Raster to Polygon e rasterize

No R para realizar a conversão de raster para polígonos e vice-versa, podemos utilizar o mesmo pacote `raster` em conjunto com `sf`. Entretanto, diferente do Python no R é bem mais fácil, podemos realizar a operação utilizando somente uma função.

```
poligonos <- rasterToPolygons(rasterReclassificado, fun=function(x){x==1},
                               na.rm=TRUE)
head(poligonos)

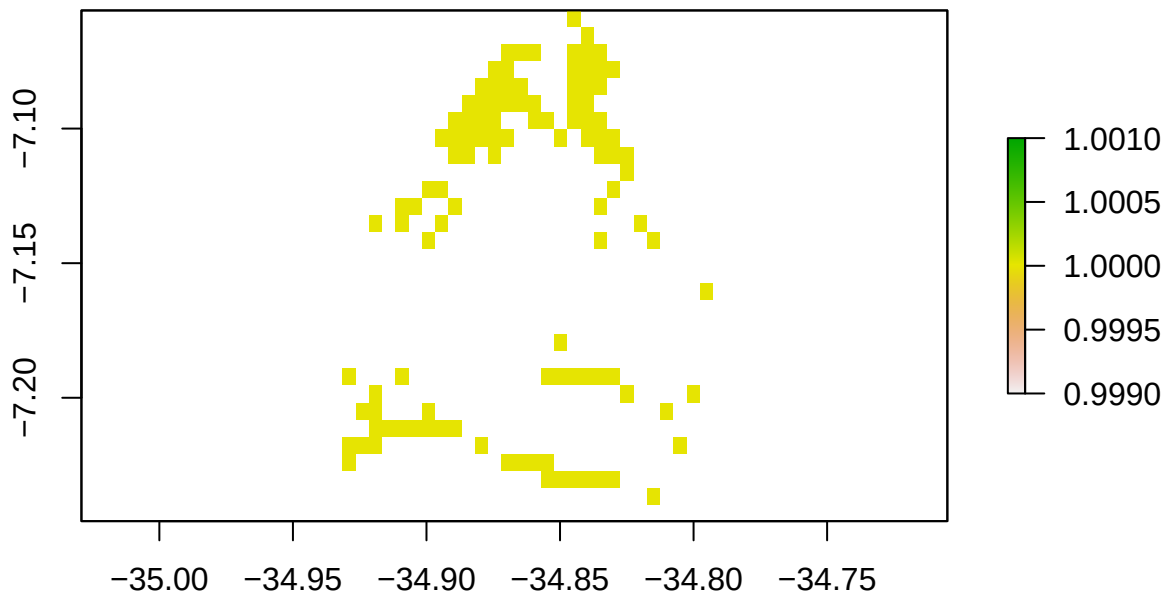
##      X07S36_ZN
## 1           1
## 2           1
## 3           1
## 4           1
## 5           1
## 6           1

# Criar um raster vazio com a mesma extensão e resolução que o original
raster_template <- raster(extent(poligonos), nrow=30, ncol=30)
crs(raster_template) <- CRS("+proj=longlat +datum=WGS84")

raster_from_polygons <- rasterize(poligonos, raster_template, field=1)

plot(raster_from_polygons, main="Soma Constante", xlim=c(xmin, xmax), ylim=c(ymin, ymax))
```

Soma Constante



Análise Espacial

Para iniciar uma análise espacial abrangente utilizando dados do Open Street Map (OSM) através da API Geofabrik, focada na região Nordeste do Brasil, no ambiente de programação R, podemos seguir uma abordagem sistemática que envolve várias etapas essenciais. Essas etapas nos permitirão extrair, processar e analisar dados geoespaciais, utilizando as capacidades do R para isso.

```
# URL do arquivo shapefile da região Nordeste
#url <- "https://download.geofabrik.de/south-america/brazil/nordeste-latest-free.shp.zip"
#nome_arquivo <- "nordeste-latest-free.shp.zip"

# Baixar o arquivo shapefile
#options(timeout=300)
#download.file(url, destfile = nome_arquivo, mode = "wb", )
```

Após o download, utilizaremos a função `unzip` para extrair os arquivos shapefile, preparando-os para a leitura e manipulação no R.

```
# Descompactar o arquivo
#nome_arquivo <- "nordeste-latest-free.shp.zip"
#unzip(nome_arquivo, exdir = "D:/Pos-GeoDataScience/M2-R/dado_pratica/nordeste-latest-free")
```

Com os dados descompactados, procederemos à leitura dos shapefiles usando a função `st_read`, como fizemos anteriormente.

```
# Caminho para o arquivo shapefile desejado
caminho_shapefile <- "D:/Pos-GeoDataScience/M2-R/dado_pratica/nordeste-latest-free/gis_osm_pois_free_1.

# Ler o shapefile
dados_osm <- st_read(caminho_shapefile)
```

```
## Reading layer 'gis_osm_pois_free_1' from data source
## 'D:\Pos-GeoDataScience\M2-R\dado_pratica\nordeste-latest-free\gis_osm_pois_free_1.shp'
## using driver 'ESRI Shapefile'
## Simple feature collection with 76616 features and 4 fields
## Geometry type: POINT
## Dimension: XY
## Bounding box: xmin: -48.62605 ymin: -18.34841 xmax: -29.34593 ymax: 0.9168696
## Geodetic CRS: WGS 84
```

```
restaurantes <- dados_osm %>% filter(fclass == "restaurant")
restaurantes <- st_transform(restaurantes, crs = 3857)
head(restaurantes)
```

```
## Simple feature collection with 6 features and 4 fields
## Geometry type: POINT
## Dimension: XY
## Bounding box: xmin: -4345163 ymin: -1666230 xmax: -3918469 ymax: -651212
## Projected CRS: WGS 84 / Pseudo-Mercator
##      osm_id code      fclass      name
## 1 430453305 2301 restaurant      Da Giovanni
## 2 501152457 2301 restaurant  Restaurante da APURN
## 3 501152491 2301 restaurant Restaurante da reitoria UFRN
## 4 501195378 2301 restaurant      Mirassol Lanches
## 5 501195402 2301 restaurant      Con Xin China
## 6 501195411 2301 restaurant      Bari Palesi
##      geometry
## 1 POINT (-4345163 -1666230)
## 2 POINT (-3918469 -651212.9)
## 3 POINT (-3918620 -651212)
## 4 POINT (-3919484 -651557.8)
## 5 POINT (-3918845 -652528.6)
## 6 POINT (-3918826 -652545.7)
```

Seleção Espacial

Para realizar uma seleção espacial em R, podemos utilizar `st_within`. Este processo envolve filtrar um conjunto de dados espaciais, como pontos de interesse (por exemplo, restaurantes), com base na sua localização dentro de uma área específica (por exemplo, um município). Vamos também ajustar os sistemas de referência de coordenadas (CRS) para garantir a compatibilidade entre os conjuntos de dados.

Primeiro, precisamos garantir que ambos os conjuntos de dados, os pontos de interesse e a área de interesse (município específico), estejam no mesmo sistema de referência de coordenadas para realizar operações espaciais precisas.

```
# Carregar dados geoespaciais
municipios <- st_read("D:/Pos-GeoDataScience/M2-R/dado_pratica/PB_Municipios/PB_Municipios_2022.shp")
```

```
## Reading layer 'PB_Municipios_2022' from data source
## 'D:\Pos-GeoDataScience\M2-R\dados_praticas\PB_Municipios\PB_Municipios_2022.shp'
## using driver 'ESRI Shapefile'
## Simple feature collection with 223 features and 4 fields
## Geometry type: MULTIPOLYGON
## Dimension: XY
## Bounding box: xmin: -38.7656 ymin: -8.302955 xmax: -34.79309 ymax: -6.025912
## Geodetic CRS: SIRGAS 2000
```

```
joaoPessoa <- municipios %>% filter(municipios$NM_MUN == 'João Pessoa')
joaoPessoa <- st_transform(joaoPessoa, crs = 3857)
```

Agora, podemos usar a função `st_within` do pacote `sf` para realizar a seleção espacial, identificando os restaurantes que estão dentro do município específico.

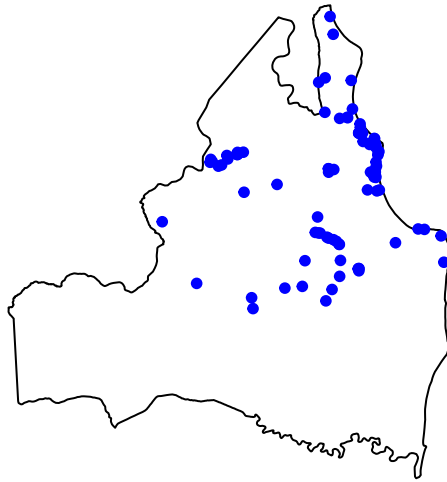
```
# Realizar a seleção espacial
indices <- st_within(restaurantes, joaoPessoa, sparse = FALSE)

# Filtrar os restaurantes que estão dentro de João Pessoa
restaurantes_joaoPessoa <- restaurantes[indices, ]
```

```
head(restaurantes_joaoPessoa)
```

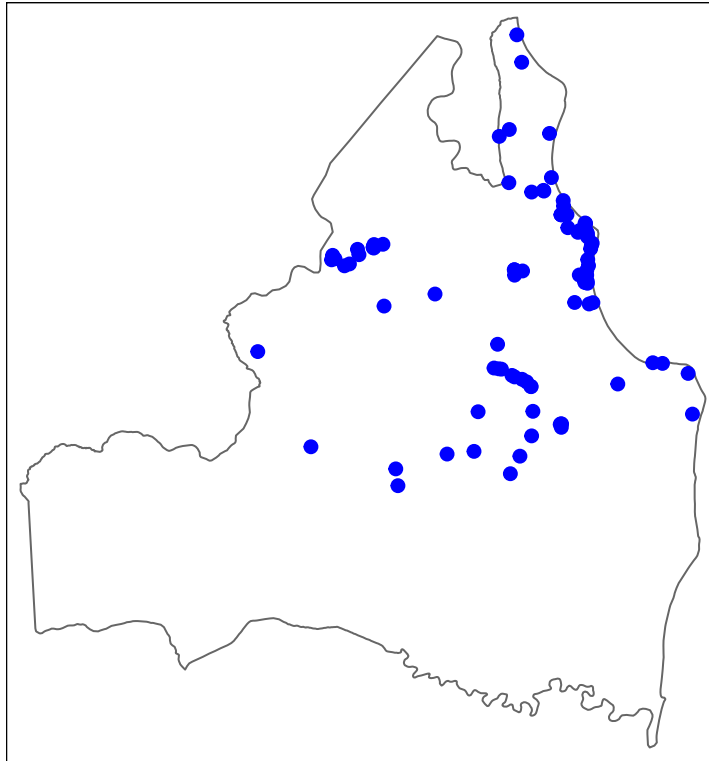
```
## Simple feature collection with 6 features and 4 fields
## Geometry type: POINT
## Dimension: XY
## Bounding box: xmin: -3877288 ymin: -797530.4 xmax: -3874684 ymax: -792958.1
## Projected CRS: WGS 84 / Pseudo-Mercator
##      osm_id code    fclass      name
## 246 1330000526 2301 restaurant Corais Bar e Restaurante
## 248 1351960638 2301 restaurant   Kyoto Restaurante
## 515 1957132650 2301 restaurant   Nau Frutos do Mar
## 529 2014595894 2301 restaurant   Parmegiana
## 536 2022094181 2301 restaurant   Monkey
## 555 2052041597 2301 restaurant   Adega do Alfredo
##      geometry
## 246 POINT (-3874684 -797530.4)
## 248 POINT (-3876669 -795189.1)
## 515 POINT (-3877163 -793590.6)
## 529 POINT (-3876812 -793670.1)
## 536 POINT (-3876585 -793860.3)
## 555 POINT (-3877288 -792958.1)
```

```
# Usando o pacote sf para plotagem simples
plot(st_geometry(joaoPessoa))
plot(st_geometry(restaurantes_joaoPessoa), add = TRUE, col = 'blue', pch = 20)
```



```
#plotando com tmap  
tm_shape(joaoPessoa) +  
  tm_borders() +  
  tm_shape(restaurantes_joaoPessoa) +  
  tm_dots(col = "blue", size = 0.2) +  
  tm_layout(main.title = "Restaurantes em João Pessoa")
```


Restaurantes em João Pessoa



Join

Para realizar a operação de join, podemos usar o pacote `sf` para trabalhar com dados espaciais e o pacote `dplyr` para manipulação de dados.

```
# Ler dados de faturamento dos restaurantes
restaurante_faturamento <- read_csv("D:/Pos-GeoDataScience/M2-R/dado_pratica/restaurantes.csv")
```

```
## New names:
## Rows: 89 Columns: 3
## -- Column specification
## ----- Delimiter: "," dbl
## (3): ...1, osm_id, fat
## i Use 'spec()' to retrieve the full column specification for this data. i
## Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## * ' -> '...1'
```

```
# Certificar-se de que os IDs estão no mesmo tipo para o join
restaurantes_joaoPessoa$osm_id <- as.integer(restaurantes_joaoPessoa$osm_id)
```

```
## Warning: NAs introduced by coercion to integer range
```

```
restaurante_faturamento$osm_id <- as.integer(restaurante_faturamento$osm_id)
```

```
## Warning: NAs introduced by coercion to integer range
```

```
# Realizar o join pelo ID
```

```
lojas_com_atributos <- inner_join(restaurantes_joaoPessoa, restaurante_faturamento, by = "osm_id")
```

```
## Warning in sf_column %in% names(g): Detected an unexpected many-to-many relationship between 'x' and  
## i Row 14 of 'x' matches multiple rows in 'y'.  
## i Row 14 of 'y' matches multiple rows in 'x'.  
## i If a many-to-many relationship is expected, set 'relationship =  
## "many-to-many"' to silence this warning.
```

```
head(lojas_com_atributos$fat)
```

```
## [1] 1496483 324589 894345 575350 1446024 618100
```

Distância

No R também podemos realizar o cálculo da matriz de origem destino ou matriz de distâncias entre pares de pontos, utilizaremos o pacote `sf` para trabalhar com dados espaciais e a função `st_distance` para calcular a distância entre pontos.

```
# Calcular a matriz de distância
```

```
matriz_distancia <- st_distance(head(lojas_com_atributos))
```

```
# Exibir a matriz de distância
```

```
print(matriz_distancia)
```

```
## Units: [m]
```

```
##           1           2           3           4           5           6  
## 1      0.000 3069.848 4655.0917 4408.3812 4133.1898 5261.8397  
## 2 3069.848      0.000 1673.1055 1525.7975 1331.5264 2315.1766  
## 3 4655.092 1673.105      0.0000 359.5948 638.3080 644.6475  
## 4 4408.381 1525.797 359.5948      0.0000 296.8024 856.0248  
## 5 4133.190 1331.526 638.3080 296.8024      0.0000 1143.8075  
## 6 5261.840 2315.177 644.6475 856.0248 1143.8075      0.0000
```

Área

Para realizar o processamento de cálculos de áreas utiliza-se a função `st_area`. Aqui neste exemplo aplicaremos um buffer em torno de cada ponto e combinando esses buffers em uma única área contínua para avaliar a cobertura ou o impacto de entidades específicas, como lojas ou serviços, em uma área determinada.

```
# Criar buffers de 30 metros ao redor de cada ponto
```

```
buffers <- st_buffer(lojas_com_atributos, dist = 1000, crs = 3857)
```

```
# Dissolver os buffers em um único polígono
```

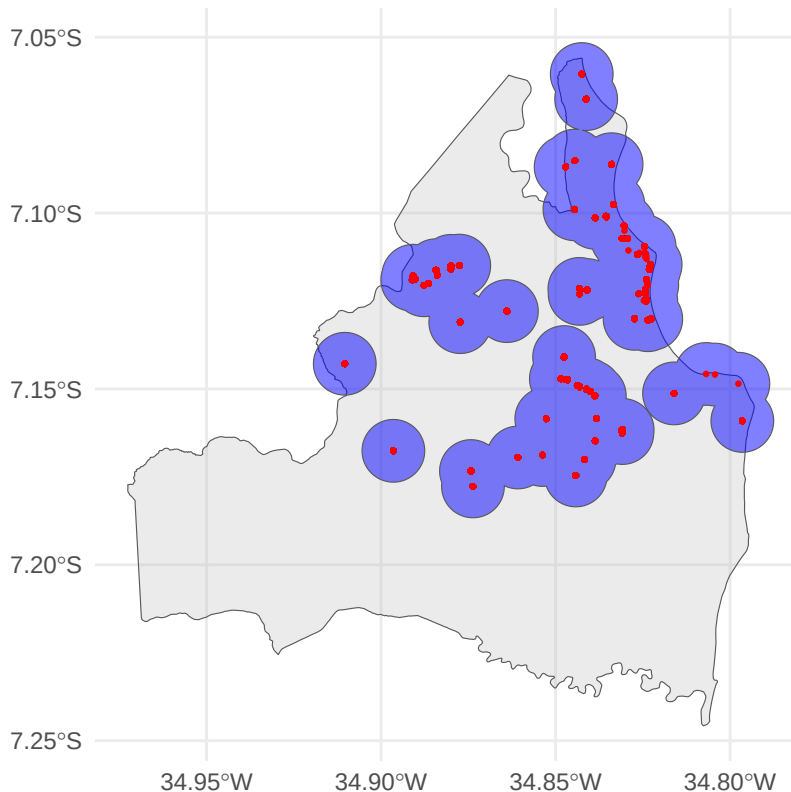
```
buffer_dissolvido <- st_union(buffers)
```

```
# Calcular a área do buffer dissolvido
area_buffer <- st_area(buffer_dissolvido, check_ring_order = FALSE)
print(paste("Área do Buffer Dissolvido:", area_buffer, "m²"))
```

```
## [1] "Área do Buffer Dissolvido: 78243493.1039168 m²"
```

```
# Plot com a delimitação do município, buffer e lojas
ggplot() +
  geom_sf(data = joaoPessoa, fill = 'grey', alpha = 0.3, size = 0.5) + # Delimitação do município
  geom_sf(data = buffer_dissolvido, fill = 'blue', alpha = 0.5) + # Buffer ao redor das lojas
  geom_sf(data = lojas_com_atributos, color = 'red', size = 0.5) + # Lojas
  theme_minimal() +
  ggtitle("Buffer ao Redor das Lojas com Delimitação do Município")
```

Buffer ao Redor das Lojas com Delimitação do Município



Sumarizar

Outro processo importante é a sumarização espacial, que permite agrupamentos baseados em atributos específicos. Esse processo envolve a agregação de informações de entidades geoespaciais que se intersectam, fornecendo insights significativos por meio de métricas como a contagem de pontos, o valor máximo, média, o comprimento total de linhas, ou a área combinada de polígonos que se sobrepõem.

```
setores <- st_read("D:/Pos-GeoDataScience/M2-R/dado_pratica/PB_Setores/PB_Setores_2021.shp")
```

```
## Reading layer 'PB_Setores_2021' from data source
## 'D:\Pos-GeoDataScience\M2-R\dado_pratica\PB_Setores\PB_Setores_2021.shp'
## using driver 'ESRI Shapefile'
## Simple feature collection with 9269 features and 12 fields
## Geometry type: MULTIPOLYGON
## Dimension: XY
## Bounding box: xmin: -38.7656 ymin: -8.302955 xmax: -34.79309 ymax: -6.025912
## Geodetic CRS: SIRGAS 2000
```

```
setores_proj <- st_transform(setores, crs = 3857)
lojas_com_atributos <- st_transform(lojas_com_atributos, crs = 3857)
```

```
lojas_em_setores <- st_join(lojas_com_atributos, setores_proj, join = st_within)
```

```
sumarizacao <- lojas_em_setores %>%
  group_by(CD_SETOR) %>%
  summarise(
    quantidade_lojas = n(),
    media_faturamento = mean(fat, na.rm = TRUE),
    faturamento_maximo = max(fat, na.rm = TRUE),
    faturamento_minimo = min(fat, na.rm = TRUE)
  )
```

```
print(sumarizacao)
```

```
## Simple feature collection with 59 features and 5 fields
## Geometry type: GEOMETRY
## Dimension: XY
## Bounding box: xmin: -3886210 ymin: -801117.2 xmax: -3873528 ymax: -787963.5
## Projected CRS: WGS 84 / Pseudo-Mercator
## # A tibble: 59 x 6
##   CD_SETOR      quantidade_lojas media_faturamento faturamento_maximo
##   <chr>          <int>          <dbl>          <dbl>
## 1 250750705000043           76      763274.      1499753
## 2 250750705000072          152      763274.      1499753
## 3 250750705000073           76      763274.      1499753
## 4 250750705000076           76      763274.      1499753
## 5 250750705000143           76      763274.      1499753
## 6 250750705000144          152      763274.      1499753
## 7 250750705000149          152      763274.      1499753
## 8 250750705000162           76      763274.      1499753
## 9 250750705000163          152      763274.      1499753
## 10 250750705000168          152      763274.      1499753
## # i 49 more rows
## # i 2 more variables: faturamento_minimo <dbl>, geometry <GEOMETRY [m]>
```

```
plot(sumarizacao)
```

